

Course Outline: React & Next.js Fundamentals — Props, Hooks, and Modularization

Title: React & Next.js Fundamentals — Props, Hooks, and Modularization **Complexity:** Normal **Prerequisite:** Week 5, Day 14 — Identifying and Defining Components of a Web Application **Target Audience:** Beginner developers who have been introduced to React/Next.js components, the App Router, and basic SPA architecture, and are now ready to build interactive, modular UIs **Total Lessons:** 22 **Format:** Mixed (36% hands-on = 8 lessons)

Course Description

You've already built your first React components and understood how Next.js organizes routes and pages. Now it's time to make those components actually talk to each other, react to data, and scale gracefully. This course takes you from static component trees to dynamic, interactive interfaces by teaching the three pillars that make React powerful: props, hooks, and modularization.

You'll learn how to pass data between components using props — including the special `children` prop that unlocks true composability. You'll then explore the hook pattern, React 19's unified `use` hook, and how to extract reusable logic into custom hooks that keep your codebase clean and DRY. You'll also master the Next.js-specific hooks that let you read the current route and query parameters. Finally, you'll learn how to control when and why React re-renders, and how to organize your project so it can grow without becoming a maze.

By the end of this course, you'll be able to build a multi-component Next.js application where data flows predictably, logic is reusable, and every file has a clear purpose.

Course Philosophy

This course emphasizes:

- **Iteration over perfection:** Build working components first, then extract and refactor as patterns emerge
 - **Path of least resistance:** Learn the simplest hook before reaching for complex solutions
 - **Human-in-the-loop:** Use AI to scaffold components, but read and verify every prop and hook before trusting the output
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Props	4
02 - The Hook Pattern	4
03 - Next.js Hooks in Action	3
04 - Controlling Rendering	4
05 - Frontend Modularization	3
06 - Assessment	2
07 - Code Challenge	1
Total	22

Section 00: Welcome

00.0 Welcome: From Components to Interactive UIs 📖

Learning Objectives:

- Identify what was covered in the previous learnpack and how this course builds on it
- Describe the three main topics this course covers: props, hooks, and modularization
- Set realistic expectations for what you'll be able to build by the end

Content Outline:

- Quick recap: what a component is, what the App Router does, and where we left off
- The gap between "static components" and "interactive UIs" — what's missing
- Roadmap of this course: props → hooks → rendering control → modularization
- What you'll build in the Code Challenge

Transitions: This lesson sets the stage. The next section starts with props — the simplest mechanism for making components communicate.

Key Principles:

- Components alone are just HTML factories; props and hooks are what make them dynamic
- Good React code is predictable: data flows one way, logic is isolated, and files have one job

Examples:

- A `ProductCard` component that always shows the same product is useless — it needs props to show any product
 - A `useAuth` custom hook lets five different pages check login status without duplicating a single line
-

Section 01: Props — Passing Data Between Components

01.0 What Are Props and How Do They Work? 📖

Learning Objectives:

- Define what props are and explain their role in the React component model
- Describe how data flows from parent to child through props
- Identify the correct TypeScript syntax for defining and receiving props
- Distinguish between required and optional props

Content Outline:

- Props as the "input parameters" of a component — the analogy to function arguments
- How a parent passes props: `<Button label="Submit" disabled={false} />`
- How a child receives props: destructuring vs the `props` object
- Typing props with TypeScript interfaces
- Required vs optional props (`?` operator and default values)
- What happens if a required prop is missing (TypeScript error at compile time)

Transitions: Now that you know how to pass simple values, the next lesson covers the special `children` prop — which lets you pass entire JSX trees into a component.

Key Principles:

- Props are read-only inside the child — a component never modifies its own props
- Data always flows downward: parent → child, never the other way
- Typing your props catches bugs before the browser ever runs your code

Examples:

- `<Avatar name="María" size="large" />` — the `Avatar` component receives and renders both values
 - A `Button` component with `label: string` (required) and `disabled?: boolean` (optional, defaults to `false`)
 - Forgetting to pass `label` to `Button` triggers a TypeScript error: "Property 'label' is missing"
-

01.1 The Children Prop — Composing Components

Learning Objectives:

- Explain what the `children` prop is and why it exists
- Use the `children` prop to build wrapper and layout components
- Identify the TypeScript type for `children` (`React.ReactNode`)
- Describe the difference between a component that receives data props vs one that receives `children`

Content Outline:

- The problem `children` solves: components that need to wrap other components
- Syntax: `<Card><p>Hello</p></Card>` — everything between the tags becomes `children`
- Receiving `children` in a component: `{ children }: { children: React.ReactNode }`
- Real use cases: layout wrappers, modal containers, card shells, section wrappers
- When to use `children` vs when to use a named prop (e.g., `content={<p>...</p>}`)

Transitions: You've seen how to pass data and JSX into components. The next lesson puts both to work with hands-on practice.

Key Principles:

- `children` enables composition — building complex UIs by nesting simple components
- A layout component shouldn't know what's inside it; it just positions the `children`
- Prefer `children` over a `content` prop when the content is free-form JSX

Examples:

- A `Card` component that wraps any content in a white box with a shadow — it doesn't care what's inside
 - A `PageLayout` component with a fixed `<Navbar />` at the top and `{children}` filling the rest
 - A `Modal` component: `<Modal><ContactForm /></Modal>` — the modal handles the overlay, the form handles the content
-

01.2 Common Props Patterns in Practice

Learning Objectives:

- Write a component that receives and uses at least three different props
- Apply the `children` prop to build a reusable wrapper component
- Use TypeScript to type all props correctly with required and optional fields
- Pass props from a parent page component down to two levels of children

Content Outline:

- Exercise setup: a simple product listing page
- Step 1: Build a `ProductCard` component with typed props (`name`, `price`, `imageUrl`, `inStock?`)
- Step 2: Build a `SectionWrapper` component that uses `children` and a `title` prop
- Step 3: Use both components in a `HomePage` to render a list of three products inside the section wrapper
- Inspecting props in React DevTools (brief intro)

Transitions: You can now build and compose components using props. The next lesson shows the most common props mistakes — so you know what to watch for in your own code and in AI-generated code.

Key Principles:

- Build the interface first, then decide what props are needed — don't over-engineer from the start
- Always check AI-generated prop interfaces: they often make everything optional when some fields should be required

Hands-On Exercise: Build a `ProfileCard` component that accepts `name: string`, `role: string`, `bio?: string`, and `children: React.ReactNode`. Render it from a parent page passing all four values. Then render a second `ProfileCard` without `bio` — confirm it still renders gracefully.

Code Example:

```
interface ProfileCardProps {
  name: string;
  role: string;
  bio?: string;
  children: React.ReactNode;
}

export function ProfileCard({ name, role, bio, children }: ProfileCardProps) {
  return (
    <div className="card">
      <h2>{name}</h2>
      <p>{role}</p>
      {bio && <p>{bio}</p>}
      <div>{children}</div>
    </div>
  );
}
```

Success Criteria:

- `ProfileCard` renders correctly with and without `bio`
- TypeScript shows an error if `name` or `role` is omitted
- `children` renders whatever JSX is passed between the tags

01.3 Anti-Pattern: Prop Drilling and the Overloaded Prop

Learning Objectives:

- Define prop drilling and explain why it becomes a problem at scale
- Identify the "overloaded prop" anti-pattern in a component interface
- Describe at least two warning signs that a component is receiving too many props
- Apply the correct refactoring strategy for each anti-pattern

Content Outline:

- **Prop Drilling:** passing props through multiple intermediate components that don't use them
 - Why it happens: a grandchild needs data from a grandparent
 - The cost: every intermediate component becomes coupled to data it doesn't care about
 - The fix: restructure the component tree or (later in the curriculum) use a shared store
- **The Overloaded Prop:** a single component with 10+ props that tries to do everything
 - Example: `<Button label onClick onHover size color variant icon disabled loading />` — this is a sign to split the component
 - The fix: break the component into smaller, more focused ones
- **Props as config objects:** passing a single object prop instead of many individual ones — when it helps and when it hides the real problem

Transitions: Props handle data flowing into a component. The next section introduces hooks — which handle state and side effects inside a component.

Key Principles:

- If a component passes a prop only to pass it further down, that's a red flag
- A component interface with more than 5–6 props usually signals a design problem
- Anti-patterns aren't always wrong at first — they become wrong as the codebase grows

Examples:

- `App` passes `user` → `Layout` → `Sidebar` → `UserAvatar` — only `UserAvatar` uses `user`. That's prop drilling.
 - A `Form` component with `onSubmit`, `onCancel`, `onReset`, `onValidate`, `onError`, `submitLabel`, `cancelLabel`, `isLoading`, `isDisabled` — split this into `Form`, `FormActions`, and `FormField`
 - AI-generated components almost always prop-drill; learn to spot it in the first review pass
-

Section 02: The Hook Pattern

02.0 What Is a Hook and Why Does It Exist?

Learning Objectives:

- Explain the problem hooks solve compared to writing logic directly in a component
- Describe the two rules of hooks (only call at the top level; only call from React functions)
- Distinguish between built-in hooks and custom hooks
- Identify when a piece of logic is a good candidate to become a hook

Content Outline:

- Before hooks: the problem of logic scattered across components (repeated fetch calls, repeated state declarations)
- What a hook is: a function whose name starts with `use` that can call other hooks
- The two rules of hooks — and what breaks when you violate them
- Built-in hooks (overview, no deep dive yet): `use`, `useContext`, `useRef`, `useCallback`, `useMemo`
- Custom hooks: extracting reusable logic into a named function
- The mental model: a hook is a "logic capsule" — it has no JSX, but it can hold state, run effects, and return values

Transitions: Now that you understand what hooks are for, the next lesson introduces the `use` hook — React 19's unified replacement for `useState` and `useEffect`.

Key Principles:

- Hooks don't render anything — they just provide logic and data to the component that calls them
- The `use` prefix isn't decoration; it's a contract that tells React this function follows the hook rules
- If you find yourself copying the same `useState` + fetch block into three components, it's time for a custom hook

Examples:

- Without hooks: three different components each have `const [user, setUser] = useState(null)` and a `useEffect` to fetch the user — identical code in three places
 - With a custom hook: `const { user, isLoading } = useCurrentUser()` — one function, used in three places
 - Violating rule 1: calling `useEffect` inside an `if` block — React throws an error because hook call order must be stable
-

02.1 The `use` Hook — React 19's Unified State and Effects

Learning Objectives:

- Explain how the `use` hook replaces `useState` and `useEffect` in React 19
- Write a component that uses `use` to manage a piece of local state
- Write a component that uses `use` to fetch data and handle the loading/error states
- Compare the `use` hook syntax to the older `useState` / `useEffect` pattern

Content Outline:

- Quick context: `useState` and `useEffect` were introduced in React 16.8 — still valid, still everywhere in codebases and AI-generated code. The curriculum teaches `use` as the current standard.
- What `use` does: reads the value of a resource (a Promise or a Context) and integrates with React's Suspense
- Managing state with `use`: `const [count, setCount] = use(State(0))` — analogous to `useState`
- Handling async data with `use`: passing a Promise to `use` — React suspends the component until the Promise resolves
- The `useTransition` pairing: wrapping state updates that might be slow
- Reading the older `useState` / `useEffect` pattern so you can understand existing code and AI output

Transitions: You can now use the `use` hook to manage state and async data. The next lesson shows how to package that logic into a reusable custom hook.

Key Principles:

- `use` is designed for the React 19+ world, but you will encounter `useState` / `useEffect` everywhere — know both
- Never call `use` conditionally — it must be called on every render in the same order
- Separating data fetching from rendering is the goal: the component declares what it needs, the hook figures out how to get it

Examples:

- `const [isOpen, setIsOpen] = use(State(false))` — a toggleable modal state
 - A `UserProfile` component that passes a `fetchUser(id)` Promise to `use` — React renders a Suspense fallback while the Promise is pending
 - Comparing old and new: the same "fetch and display" feature written with `useEffect` vs `use` — same behavior, different syntax
-

02.2 Building Your First Custom Hook

Learning Objectives:

- Extract repeated stateful logic from a component into a named custom hook
- Define the input parameters and return values of a custom hook
- Apply the `use` prefix naming convention correctly
- Consume a custom hook in at least two different components and verify it works independently in each

Content Outline:

- When to extract a custom hook: the "copy-paste" signal — if you're about to duplicate logic, extract it first
- Anatomy of a custom hook: a function that starts with `use`, can call other hooks, and returns data or functions
- Step-by-step: building `useToggle` from scratch
- Step-by-step: building `useWindowWidth` to read the browser window size
- Custom hooks always return something: an array (like `useState`), an object, or a single value
- Testing that the hook works independently in two components

Transitions: You've built reusable logic with custom hooks. The next lesson shows the most damaging hook mistakes — many of which come directly from AI-generated code.

Key Principles:

- A custom hook has no JSX — it is purely logic
- Always return from a custom hook; a hook that returns nothing is usually a sign it should be a utility function instead
- Custom hooks should have a single, clear responsibility — `useAuth`, not `useAuthAndNavigationAndTheme`

Hands-On Exercise: Build a `useCounter` custom hook that accepts an `initialValue: number` and returns `{ count, increment, decrement, reset }`. Use it in two separate components on the same page: one that counts items added to a cart, and one that counts the number of times a video has been played. Confirm that each component's count is independent.

Code Example:

```
// hooks/useCounter.ts
import { use } from "react";

export function useCounter(initialValue: number = 0) {
  const [count, setCount] = use(State(initialValue));
  const increment = () => setCount((c) => c + 1);
  const decrement = () => setCount((c) => c - 1);
  const reset = () => setCount(initialValue);
  return { count, increment, decrement, reset };
}
```

Success Criteria:

- `useCounter` lives in its own file under `/hooks`
- Both components use `useCounter` independently — changing count in one does not affect the other
- TypeScript correctly infers all return types

02.3 Anti-Pattern: Misusing Hooks and Effect Storms

Learning Objectives:

- Identify the three most common hook misuse patterns in React codebases
- Explain what an "effect storm" is and describe how it happens
- Recognize these patterns in AI-generated code during a review pass
- Apply the correct fix for each anti-pattern

Content Outline:

- **Effect Storm (useEffect without dependency control):** a `useEffect` that re-runs on every render because its dependency array is missing or wrong — it triggers a state update, which causes a re-render, which triggers the effect again
 - The fix: explicitly list dependencies; never leave the array empty unless the effect should truly run once
- **God Hook:** a custom hook that manages state, fetches data, handles errors, updates the URL, and logs analytics — all in one function
 - The fix: split by responsibility — `useFetch`, `useErrorHandler`, `useAnalytics`
- **Hook inside a condition or loop:** calling `useEffect` or `use` inside an `if` block or `for` loop — breaks React's internal hook order tracking
 - The fix: move the hook call to the top level and handle the condition inside the hook
- Note: AI agents frequently generate effect storms when connecting data to components — always check the dependency array first

Transitions: You now have a solid understanding of hooks and their failure modes. The next section moves to Next.js-specific hooks that let your components interact with the URL.

Key Principles:

- An effect that triggers a state update that triggers the same effect is always a bug
- One hook, one responsibility — the same Single Responsibility Principle that applies to components applies to hooks
- Read the dependency array every time you see a `useEffect` in AI output — it's the most commonly wrong line

Examples:

- Effect storm: `useEffect(() => { setData(fetch(...)) })` with no dependency array — runs on every render, sets state, triggers re-render, repeats infinitely
- God hook: `useUserDashboard` that fetches user data, tracks page views, manages a modal state, and handles logout — split into four focused hooks
- Hook in condition: `if (isLoggedIn) { useEffect(...) }` — React throws: "Rendered more hooks than during the previous render"

Section 03: Next.js Hooks in Action

03.0 `usePathname` and `useSearchParams` — Navigating the App

Learning Objectives:

- Explain what `usePathname` returns and when to use it
- Explain what `useSearchParams` returns and how to read values from it
- Identify the `"use client"` directive requirement for these hooks
- Describe two common UI patterns that depend on the current pathname

Content Outline:

- Why Next.js needs its own hooks: the server/client boundary means standard browser APIs aren't always available
- `usePathname` : returns the current URL path as a string (e.g., `/products/shoes`)
 - Common use: highlighting the active link in a navbar
 - Syntax: `const pathname = usePathname()`
- `useSearchParams` : returns a read-only `URLSearchParams` object
 - Reading a value: `searchParams.get("category")` → `"shoes"`
 - Common use: reading filter/sort parameters from the URL
- The `"use client"` directive: these hooks only work in client components — what that means and why
- Relationship to the App Router: how these hooks connect to the folder-based routing covered in Day 14

Transitions: Now that you know how to read the URL, the next lesson goes deeper into query strings — what they are, how they're structured, and how to use them for real UI features.

Key Principles:

- The URL is a piece of shared state — any component can read it without prop drilling
- Always add `"use client"` at the top of any file that uses `usePathname` or `useSearchParams`
- Never mutate `searchParams` directly — it's read-only; use Next.js navigation functions to update the URL

Examples:

- A `Navbar` component that uses `usePathname` to add an `active` class to the current page's link
- A `ProductList` that uses `useSearchParams` to read `?category=shoes&sort=price` and filter the displayed items
- Forgetting `"use client"` causes a build error: `"usePathname only works in Client Components"`

03.1 What Are Query Strings and How to Use Them?

Learning Objectives:

- Define what a query string is and describe its structure in a URL
- Read one or more query string parameters using `useSearchParams`
- Explain why query strings are preferred over component state for shareable UI state
- Identify two use cases where query strings are the right tool

Content Outline:

- Anatomy of a URL: `https://store.com/products?category=shoes&sort=price&page=2`
 - The `?` separator, `key=value` pairs, & between pairs
- Why query strings matter for UX: a URL with query strings is shareable and bookmarkable — the same UI state is reproducible from just the URL
- Reading query strings with `useSearchParams` :
 - `.get("key")` → single value or `null`
 - `.getAll("key")` → array of values (for multi-select filters)

- `.has("key")` → boolean check
- When query strings are better than component state: filters, search terms, pagination, active tab
- When to use component state instead: transient UI state like "is this dropdown open?" — something the user doesn't need to share or bookmark

Transitions: You understand what query strings are and why they exist. The next lesson puts `usePathname` and `useSearchParams` to work in a hands-on exercise.

Key Principles:

- If the user would be frustrated to lose it on page refresh, it belongs in the URL, not in state
- Query strings are always strings — parse numbers and booleans explicitly (`parseInt` , `=== "true"`)
- Never store sensitive data in query strings — they appear in browser history and server logs

Examples:

- A search page: `?query=laptop&minPrice=500&maxPrice=1500` — the user can share this exact search with a colleague
- Pagination: `?page=3` — refreshing the page returns to page 3, not page 1
- A tab component: `?tab=reviews` — the active tab is bookmarkable and can be linked to directly

03.2 Working with Next.js Hooks

Learning Objectives:

- Implement a `Navbar` that highlights the active link using `usePathname`
- Read at least two query parameters from the URL using `useSearchParams`
- Display different UI content based on a query string value
- Add the `"use client"` directive correctly to components that use these hooks

Content Outline:

- Exercise Part 1: Active Navbar with `usePathname`
 - Build a `Navbar` with four links: Home, Products, About, Contact
 - Use `usePathname` to compare the current path to each link's `href`
 - Apply a different style to the matching link
- Exercise Part 2: Filtered List with `useSearchParams`
 - Build a `ProductList` component that receives a hardcoded array of products
 - Read a `?category=` query param from the URL
 - Filter the displayed products to match the selected category
 - Display "All products" when no category param is present
- Running both features in the same Next.js app and testing by changing the URL manually

Transitions: You can now read and react to the URL from any client component. The next section covers a topic that trips up almost every React developer: controlling when components re-render.

Key Principles:

- Test URL-based features by typing directly in the browser address bar — no button clicks needed
- Always handle the `null` case from `.get()` — a missing query param is a valid state, not a bug

Hands-On Exercise: Add a fifth link to your `Navbar` that navigates to `/products?category=electronics` . When you land on that page, the `ProductList` should automatically filter to show only electronics. The "Products" nav link should still be highlighted since the pathname is still `/products` .

Code Example:

```
"use client";
import { usePathname } from "next/navigation";
```

```

export function Navbar() {
  const pathname = usePathname();
  const links = [
    { label: "Home", href: "/" },
    { label: "Products", href: "/products" },
    { label: "About", href: "/about" },
  ];
  return (
    <nav>
      {links.map((link) => (
        <a
          key={link.href}
          href={link.href}
          className={pathname === link.href ? "active" : ""}>
            {link.label}
          </a>
        )
      )}
    </nav>
  );
}

```

Success Criteria:

- Only the current page's link has the `active` class
- The product list filters correctly when `?category=electronics` is in the URL
- Removing the query param from the URL shows all products without a page reload

Section 04: Controlling Rendering

04.0 How React Decides to Re-Render

Learning Objectives:

- Describe the three triggers that cause a React component to re-render
- Explain the parent-child rendering relationship: when a parent re-renders, so do its children
- Identify the performance cost of unnecessary re-renders
- Describe what React's reconciler does after a re-render

Content Outline:

- The three re-render triggers: state change, prop change, parent re-render
- The cascade effect: if `App` re-renders, every component in the tree re-renders by default
- What "re-render" actually means: React calls the component function again and produces a new Virtual DOM tree
- Reconciliation: React compares the new tree to the previous one and only updates what changed in the real DOM
- Why unnecessary re-renders matter at scale: in a small app, imperceptible; in a complex UI with many components, it causes visible lag
- Practical implication: keep state as low in the tree as possible — if only the `Counter` needs it, don't lift it to `App`

Transitions: Understanding why React re-renders sets up the next lesson: CSR vs SSR, which is about a different but related question — when does React render at all (on the client vs the server)?

Key Principles:

- React always re-renders the component subtree below the changed state — understanding this tree structure is essential
- Moving state down (not up) is often the correct performance optimization

- The reconciler is smart — React touches the real DOM as little as possible, but it still has to run your component functions

Examples:

- A `Dashboard` with three widgets: if `Dashboard`'s state changes, all three widgets re-render even if none of their props changed
 - Fix: move the state into the specific widget that uses it — the other two widgets stop re-rendering
 - A `SearchInput` component that updates state on every keystroke — if it's placed too high in the tree, the whole page re-renders on each key press
-

04.1 CSR vs SSR — Choosing the Right Rendering Strategy

Learning Objectives:

- Explain the difference between Client-Side Rendering (CSR) and Server-Side Rendering (SSR) in Next.js
- Identify which rendering strategy is appropriate for a given component based on its characteristics
- Apply the `"use client"` directive correctly to designate a component as a client component
- Describe the tradeoffs of each strategy in terms of performance and interactivity

Content Outline:

- Quick recap of the concepts introduced in Day 14 — now going one level deeper
- **CSR (Client-Side Rendering):** the component runs in the browser
 - Pros: interactive, can use hooks, can access browser APIs
 - Cons: slower initial load, not indexed well by some crawlers, blank page until JS loads
 - When to use: anything with user interaction, hooks, or browser-specific behavior
- **SSR (Server-Side Rendering):** the component runs on the server, HTML is sent to the browser
 - Pros: fast initial load, better SEO, no JS required for the initial paint
 - Cons: can't use hooks or browser APIs, re-fetches on every request
 - When to use: static content, SEO-sensitive pages, data that doesn't change per user
- The `"use client"` directive: adding it marks the component and all its imports as client components
- Decision rule: default to server components; only add `"use client"` when you actually need it
- The boundary: a server component can render a client component, but not vice versa for server-specific features

Transitions: You know when to use each strategy. The next lesson covers the two most common rendering mistakes — both of which involve breaking this CSR/SSR boundary without realizing it.

Key Principles:

- In Next.js App Router, all components are server components by default — `"use client"` is opt-in
- A good rule of thumb: if the component uses a hook, it's a client component
- Don't add `"use client"` to every file — it disables server-side optimizations for that entire subtree

Examples:

- A `BlogPost` page that displays static text: server component — no hooks needed, fast load, SEO-friendly
 - A `CommentSection` with a text input and submit button: client component — needs state and event handlers
 - A `ProductPage` that has both: static product details (server) wrapping an `AddToCartButton` (client)
-

04.2 Common Rendering Mistakes and How to Fix Them

Learning Objectives:

- Identify the "renders on every parent update" anti-pattern and explain why it happens
- Explain the `useEffect` without dependency array mistake and its consequences
- Recognize these patterns in code produced by an AI agent
- Apply the correct structural fix for each mistake without using advanced memoization tools

Content Outline:

- **Mistake 1 — Re-rendering the whole parent when only one child needs to change:**
 - State lives in the parent and is unrelated to most children, but all children still re-render
 - Fix: colocate state — move it down to the component that actually uses it
- **Mistake 2 — `useEffect` without a dependency array:**
 - `useEffect(() => { setData(something) })` — no array means it runs after every render
 - This sets state, which triggers a render, which runs the effect again → infinite loop
 - Fix: identify what the effect depends on and list it explicitly; if it should run once, use `[]`
- **Mistake 3 — Rendering a child component that depends on the parent's full state object:**
 - `<ProductList products={allState} />` — any change to `allState` re-renders `ProductList`, even if `products` didn't change
 - Fix: pass only the specific data the child needs, not a large shared object
- Note: AI code frequently makes all three of these mistakes. This lesson is your code review checklist.

Transitions: Now that you can spot these mistakes, the next lesson gives you hands-on practice fixing them in a working component.

Key Principles:

- The root cause of most rendering problems is state placed too high in the component tree
- A missing dependency array in `useEffect` is one of the most common bugs in AI-generated React code — always check it
- You don't need `useMemo` or `React.memo` to fix most rendering problems; structural fixes come first

Examples:

- App has `const [theme, setTheme] = useState("dark")` — `Navbar`, `Sidebar`, and `Content` all re-render when the theme changes, even though only `Navbar` displays it
 - Fix: move `theme` state into `Navbar`
 - `useEffect(() => { fetchProducts().then(setProducts) })` — no array, runs on every render, sets state, re-renders, repeats
 - Fix: `useEffect(() => { fetchProducts().then(setProducts) }, [])` — runs once on mount
-

04.3 Controlling Re-Renders in Practice

Learning Objectives:

- Diagnose a component tree where state is placed too high and causes unnecessary re-renders
- Move state down to the correct component without breaking the application's behavior
- Fix a `useEffect` with a missing or incorrect dependency array
- Verify the fix using React DevTools Profiler (brief introduction)

Content Outline:

- Exercise setup: a broken `Dashboard` component with three problems planted deliberately
 - Problem 1: a counter state in `Dashboard` causes `Chart`, `Stats`, and `UserList` to all re-render on each count change
 - Problem 2: a `useEffect` in `UserList` has no dependency array and causes an infinite fetch loop
 - Problem 3: `Stats` receives the full `dashboardState` object when it only needs `totalRevenue`
- Step 1: identify each problem by reading the code (no tools yet)
- Step 2: move the counter state into a new `CounterWidget` component
- Step 3: fix the `useEffect` dependency array in `UserList`
- Step 4: update `Stats` to receive only `totalRevenue` as a prop
- Step 5: open React DevTools Profiler, record an interaction, and confirm the re-render count dropped

Transitions: Your components are now efficient and predictable. The final content section covers modularization — how to organize those components into a project structure that scales.

Key Principles:

- Fix structure before reaching for optimization tools — `useMemo` and `React.memo` are for performance tuning, not structural bugs
- React DevTools Profiler is the only reliable way to confirm a rendering fix actually worked

Hands-On Exercise: You are given a `Dashboard` component file (~80 lines) with the three problems described above. Fix all three without adding any new libraries. The final result should have: no state in `Dashboard` itself, a `useEffect` in `UserList` with a correct dependency array, and `Stats` receiving a single `totalRevenue: number` prop.

Code Example (before fix):

```
// ❌ State lives in Dashboard – all children re-render on count change
export function Dashboard() {
  const [count, setCount] = useState(0);
  const [dashboardData, setDashboardData] = useState(null);

  useEffect(() => { // ❌ No dependency array
    fetchDashboardData().then(setDashboardData);
  });

  return (
    <>
      <button onClick={() => setCount(c => c + 1)}>{count}</button>
      <Chart data={dashboardData} />
      <Stats data={dashboardData} /> { /* ❌ Receives full object */ }
      <UserList data={dashboardData} />
    </>
  );
}
```

Success Criteria:

- Clicking the counter button does not cause `Chart`, `Stats`, or `UserList` to re-render
- The fetch in `UserList` runs exactly once on mount
- `Stats` receives only `totalRevenue: number`, not the full data object
- React DevTools Profiler shows only `CounterWidget` highlighted when the button is clicked

Section 05: Frontend Modularization

05.0 What Is a Module and Why Split Your Code? 📖

Learning Objectives:

- Define what a module is in the context of a Next.js project
- Explain three concrete benefits of splitting code into modules
- Identify the signs that a file or component needs to be split
- Describe the relationship between modularization and the Single Responsibility Principle

Content Outline:

- What a module is: any file that exports something — a component, a hook, a utility function, a type
- Why monolithic files are a problem: hard to read, hard to test, hard to reuse, impossible to navigate
- Three signs a file needs splitting:
 1. It's longer than ~150 lines and contains multiple unrelated concerns
 2. You're importing it into a new file just for one small function it happens to contain
 3. Two different pages need the same logic, but that logic is buried inside a page component
- The Single Responsibility Principle applied to files: each file should have one clear reason to change
- Modularization is not about small files for the sake of it — it's about files that do one thing well

Transitions: Now that you know why to split, the next lesson shows exactly where to put things in a Next.js project.

Key Principles:

- A file with a single clear purpose is easier to read in 6 months — by you, by a teammate, and by an AI agent
- Don't split prematurely: write the code first, then extract when the second use case appears
- Modularization is the foundation of code that AI can maintain reliably — vague, monolithic files produce vague, broken AI edits

Examples:

- A `pages/products.tsx` that contains the page component, three sub-components, two custom hooks, and a fetch utility — this is one file doing five jobs
 - After splitting: `ProductsPage` in `/app/products/page.tsx`, `ProductCard` in `/components/ProductCard.tsx`, `useProducts` in `/hooks/useProducts.ts`
 - A `utils.ts` file with 40 unrelated functions — if you only ever need `formatCurrency`, why import the whole file?
-

05.1 Organizing Your Next.js Project — Folders That Scale

Learning Objectives:

- Identify the purpose of the most important folders in a well-organized Next.js project
- Distinguish between `/app` (routing), `/components` (UI pieces), `/hooks` (logic), `/lib` (utilities), and `/types` (TypeScript interfaces)
- Explain when to create a new folder vs when to add to an existing one
- Recognize the disorganized folder structure that AI agents commonly produce

Content Outline:

- The App Router folders (from Day 14 — not repeated): `/app` is for routing, not for housing every file
- Additional folders that every real Next.js project needs:
 - `/components` — reusable UI components (not pages)
 - `/hooks` — custom hooks
 - `/lib` or `/utils` — pure utility functions (formatters, validators, helpers)
 - `/types` — TypeScript interfaces and type aliases shared across the app
 - `/styles` — global CSS or Tailwind config (if used)
- When to introduce a subfolder: when a folder has more than 6–8 files and they fall into clear subcategories
 - Example: `/components/forms/`, `/components/layout/`, `/components/ui/`
- The AI agent problem: without a context rule or memory bank, AI agents dump everything into the root or into `/components` — your job is to enforce the structure

Transitions: You know where things go. The next lesson puts it into practice by taking a messy single-file project and splitting it correctly.

Key Principles:

- Put files where a new team member would look for them, not where they were easiest to create
- Types go in `/types`, not scattered as inline interfaces inside component files
- A `/hooks` folder communicates to every future reader (and AI agent) that reusable logic lives there

Examples:

- `/hooks/useCounter.ts` — the hook from Section 02
 - `/types/Product.ts` — `export interface Product { id: string; name: string; price: number }`
 - `/lib/formatCurrency.ts` — `export function formatCurrency(amount: number): string { return `$$${amount.toFixed(2)}` }`
-

05.2 Refactoring a Monolith Into Modules

Learning Objectives:

- Split a single large component file into correctly-located smaller files
- Move a custom hook out of a component file and into `/hooks`
- Move a TypeScript interface out of a component and into `/types`
- Import and consume the extracted modules correctly, verifying the app still works

Content Outline:

- Exercise setup: a `ShopPage` component (~120 lines) that contains:
 - The page component itself
 - A `ProductCard` sub-component defined inline
 - A `useCartCount` hook defined inline
 - A `Product` TypeScript interface defined inline
 - A `formatPrice` utility function defined inline
- Step 1: move `Product` interface to `/types/Product.ts`
- Step 2: move `formatPrice` to `/lib/formatPrice.ts`
- Step 3: move `useCartCount` to `/hooks/useCartCount.ts`
- Step 4: move `ProductCard` to `/components/ProductCard.tsx`
- Step 5: update all imports in `ShopPage` — the component behavior should be identical
- Final review: the `ShopPage` file should now be under 40 lines

Transitions: You've built a modular, well-organized React/Next.js project. The next section tests everything you've learned before the final challenge.

Key Principles:

- Refactoring means changing the structure without changing the behavior — run the app after each step to confirm nothing broke
- Each extracted file should be immediately testable in isolation
- This is the kind of refactoring AI agents struggle with — they tend to move files but break import paths

Hands-On Exercise: Given the `ShopPage` monolith described above, complete all five extraction steps. After each step, verify the app compiles and renders correctly. Final deliverable: a `/types`, `/lib`, `/hooks`, and `/components` folder each containing exactly one new file extracted from the original.

Code Example (before refactor):

```
// ❌ Everything in one file
interface Product { id: string; name: string; price: number; }

function formatPrice(price: number) { return `$$${price.toFixed(2)}`; }

function useCartCount() {
  const [count, setCount] = use(State(0));
  return { count, add: () => setCount(c => c + 1) };
}

function ProductCard({ product }: { product: Product }) {
  return <div><h3>{product.name}</h3><p>{formatPrice(product.price)}</p></div>;
}

export default function ShopPage() {
  const { count, add } = useCartCount();
  const products: Product[] = [{ id: "1", name: "Laptop", price: 999 }];
  return (
```

```

    <main>
      <p>Cart: {count}</p>
      {products.map(p => <ProductCard key={p.id} product={p} />)}
    </main>
  );
}

```

Success Criteria:

- Four new files created in the correct folders
- `ShopPage` is under 40 lines and contains only the page logic
- The app compiles and renders identically to before the refactor
- No TypeScript errors in any file

Section 06: Assessment

06.0 Debugging a React Component Tree

Learning Objectives:

- Read an unfamiliar React component tree and identify at least two structural problems
- Apply the fixes covered in this course (prop drilling, effect storms, misplaced state, missing `"use client"`)
- Explain in a comment why each fix was necessary

Content Outline:

- Exercise: a provided `NotificationsPage` component with four deliberately planted problems:
 1. Prop drilling through three levels to pass `userId`
 2. A `useEffect` without a dependency array causing an infinite loop
 3. A `usePathname` call in a server component (missing `"use client"`)
 4. A monolithic component with an inline hook and utility function
- Task: identify all four problems, fix them, and add a one-line comment above each fix explaining why it was wrong

Transitions: This exercise confirms you can read and fix real-world React problems. The next lesson tests your conceptual understanding with a quiz.

Key Principles:

- Reading code critically is as important as writing it — most of your time as a developer is spent reading
- Every fix you make is a potential AI prompt: "Fix the prop drilling in this component" works best when you already know what prop drilling is

Hands-On Exercise: Fix all four problems in the `NotificationsPage` file. Do not rewrite the entire component — make targeted, minimal edits. Each fix should be accompanied by a comment.

Success Criteria:

- No prop drilling remains — `userId` reaches its destination without passing through intermediary components
- The `useEffect` runs exactly once on mount
- The file with `usePathname` has `"use client"` at the top
- The inline hook and utility are extracted to their correct folders

06.1 Props, Hooks, and Rendering Quiz

Learning Objectives:

- Demonstrate conceptual understanding of props, hooks, rendering triggers, and modularization

- Identify correct and incorrect React/Next.js patterns from code snippets
- Apply decision rules (CSR vs SSR, state placement, hook extraction) to new scenarios

Question Format: Multiple choice (6 questions)

Questions:

1. A component receives a `user` prop but only passes it to a child without using it. What is this called?
 - a) Prop inheritance
 - b) Prop drilling ☒
 - c) Context injection
 - d) Component forwarding
2. Which of the following will cause an infinite re-render loop?
 - a) `useEffect(() => { setData(x) }, [x])`
 - b) `useEffect(() => { setData(x) }, [])`
 - c) `useEffect(() => { setData(x) })` ☒
 - d) `useEffect(() => { console.log(x) }, [x])`
3. You need to read the current URL path to highlight the active navigation link. Which hook do you use?
 - a) `useRouter`
 - b) `useSearchParams`
 - c) `usePathname` ☒
 - d) `useNavigation`
4. A component displays static blog post content that never changes. Which rendering strategy should it use in Next.js?
 - a) Client-Side Rendering (`"use client"`)
 - b) Server-Side Rendering (default, no directive needed) ☒
 - c) It doesn't matter — they produce identical results
 - d) Always use `"use client"` to be safe
5. Where should a `useCounter` custom hook be placed in a well-organized Next.js project?
 - a) `/app/hooks/useCounter.ts`
 - b) `/components/useCounter.ts`
 - c) `/hooks/useCounter.ts` ☒
 - d) Inline inside the component that uses it
6. A parent component has `const [theme, setTheme] = useState("dark")` . Only the `Navbar` component displays the theme. What is the correct fix to avoid unnecessary re-renders of the other children?
 - a) Wrap all children in `React.memo`
 - b) Move the `theme` state into the `Navbar` component ☒
 - c) Use `useMemo` on the `theme` variable
 - d) Move `theme` into a global variable outside the component

Evaluation Criteria:

- 6/6: Excellent — ready for the Code Challenge and the next learnpack
- 4–5/6: Good — review the lessons corresponding to any missed questions
- 0–3/6: Review sections 01–04 before proceeding

Section 07: Code Challenge

07.0 Build a Modular Dashboard Shell 🚀

Scenario: You are building the frontend shell for a team productivity dashboard. The app has multiple pages, and the components must be properly split, typed, and structured.

Requirements:

1. Create a Next.js project with at least three pages: `/`, `/tasks`, and `/settings`
2. Build a `Navbar` client component that highlights the active link using `usePathname`
3. Build a `PageLayout` component that accepts a `title: string` prop and a `children` prop, and wraps all page content
4. Build a `useToggle` custom hook that returns `{ isOn, toggle }` and is used by at least one component to show/hide a panel
5. Read a `?view=` query string on the `/tasks` page using `useSearchParams` and display different content based on its value (`"list"` vs `"board"`)
6. Place all custom hooks in `/hooks`, all reusable components in `/components`, and all TypeScript interfaces in `/types`
7. No component file may exceed 60 lines

Constraints:

- Use Next.js App Router
- Use TypeScript for all files
- Do not use any external state management library
- The `Navbar` and any component using `usePathname` or `useSearchParams` must have `"use client"` at the top

Expected Output / Behavior:

- Navigating between pages updates the active link highlight in the `Navbar` without a full page reload
- Visiting `/tasks?view=list` shows a list layout; visiting `/tasks?view=board` shows a board layout; visiting `/tasks` with no query param defaults to the list layout
- Clicking a toggle button on any page shows and hides a panel using the `useToggle` hook
- The project folder structure matches the conventions covered in Section 05